



4.6 Core Logic

```
def available_rooms(room_type):
    booked = sum(
        1 for b in BOOKINGS.values()
        if b["room_type"] == room_type
        and b["status"] == "Booked"
    )
    # vacancy = total rooms - active bookings
    return ROOM_TYPES[room_type]["count"] - booked

def book_room(name, room_type, nights):
    if available_rooms(room_type) <= 0:
        return None, f"No {room_type} rooms available."
    booking_id = 1001 + len(BOOKINGS) # unique, sequential
    total = ROOM_TYPES[room_type]["price"] * nights
    BOOKINGS[booking_id] = {
        "name": name, "room_type": room_type,
        "nights": nights, "total": total, "status": "Booked"
    }
    return booking_id, total
```

The `available_rooms()` function calculates the number of vacant rooms by subtracting active bookings from the total room count. The `book_room()` function checks availability, generates a unique booking ID, calculates the total cost, and stores the reservation, ensuring rooms cannot be overbooked.

4.7 Output Screenshot

```
HOTEL
BOOKING
SYSTEM

Terminal Hotel Booking System

[1] View Rooms & Availability
[2] Book a Room
[3] Check-Out / Free a Room
[4] View All Bookings
[5] Exit

> Enter option: 2
> Enter guest name: ram

SELECT ROOM TYPE:

Single - Rs.1000/night
Double - Rs.2000/night
Suite - Rs.5000/night

> Enter room type: double
> Enter number of nights: 5

ROOM BOOKED:

Booking ID : 1001
Guest      : ram
Room type  : double
Nights     : 5
Total amount: Rs.10000
```